

JAVA INTERVIEW PREPARATION

CHAPTER 41

SPRING TESTING, ACTUATOR, AND OBSERVABILITY

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring Testing, Actuator, Observability, and Production Readiness

Senior / Lead Java Developer Reading Chapter

Spring applications can appear correct in unit tests while failing because of proxy boundaries, serialization, transactions, security filters, database behavior, or auto-configuration. A senior test strategy uses the smallest environment that can prove each risk. Production readiness then exposes health, metrics, traces, logs, and controlled management operations without turning Actuator into an attack surface.

Testing and observability solve the same broad problem at different times: gaining trustworthy evidence about behavior. Tests provide controlled evidence before release; production signals reveal behavior under real data, concurrency, dependencies, and load. Neither is valuable merely because a framework annotation or dashboard exists. The evidence must correspond to a known risk and support a decision.

A mature strategy therefore starts with failure modes. If the risk is a pricing formula, a direct unit test is enough. If the risk is transaction interception, a plain unit test cannot prove it because no proxy exists. If the risk is PostgreSQL locking, an in-memory repository proves nothing. If the risk is TLS termination or probe routing, only a deployed environment may have the required fidelity.

The Testing Pyramid Is About Feedback and Fidelity

many fast tests:	ordinary unit tests
focused framework:	MVC, JSON, JPA, security, auto-config slices
fewer integration:	complete context plus real infrastructure
small end-to-end set:	deployed network behavior

The goal is not a fixed percentage. Use ordinary tests for algorithms and domain rules, and integration tests for behavior implemented by Spring or infrastructure.

The tradeoff has at least four dimensions: execution time, isolation, diagnostic clarity, and realism. Fast isolated tests usually tell us exactly which rule failed. Broad tests prove composition but have more possible causes and can become flaky when they depend on uncontrolled time, ports, or shared services.

Do not choose a test type by class name. A service class containing pure calculation needs no Spring context. A two-line repository method whose behavior depends on generated SQL and database constraints needs a database integration test. The smallest meaningful test is the one that includes the mechanism being trusted and excludes unrelated mechanisms.

Unit Tests Without Spring

Constructor injection allows direct tests:

```
@Test
void rejectsExpiredOffer() {
    PricingService service = new PricingService(repository, fixedClock);
    assertThrows(OfferExpiredException.class, () -> service.price(order));
}
```

These tests are fast, deterministic, and expose dependency design. Mock only meaningful boundaries. Excessive mocking of internal collaborators couples tests to implementation steps.

State-based tests assert the resulting value or state and usually survive refactoring better than tests that verify every internal call. Interaction verification is appropriate when the interaction is itself the contract, such as publishing exactly one outbox record, never charging after validation fails, or invoking compensation after a

partially completed workflow.

Use fakes carefully. An in-memory fake can provide deterministic domain tests, but it must not pretend to prove database isolation, query behavior, or uniqueness under concurrency. Name the evidence honestly: a fake repository proves how the service responds to repository outcomes, not how PostgreSQL produces those outcomes.

Context Tests

`@SpringBootTest` loads a broad Boot context. It can verify full wiring, properties, proxy creation, migrations, and startup, but it is expensive and can mask which feature is under test.

Context caching speeds compatible tests. Changing profiles, mock beans, dynamic properties, or dirtiness can create many distinct contexts. `@DirtiesContext` should be exceptional because it destroys cache reuse and often signals leaked mutable state.

`@SpringBootTest` uses Spring Boot to create the application context, but its default mock web environment does not start an embedded server. `RANDOM_PORT` starts a real server. This distinction matters when a test claims to cover connectors, compression, error pages, or actual network clients.

One lightweight full-context test that only verifies startup can catch missing beans, invalid configuration binding, circular dependencies, and migration initialization. It should use production-like mandatory properties rather than weakening validation for tests. A context that starts only because security, migrations, and external clients were globally disabled is not strong evidence of production readiness.

Keep context variants intentional. Each unique collection of profiles, properties, mocks, and configuration classes may create a different cache key. Hundreds of cosmetically different tests can therefore spend most of their time rebuilding almost identical applications.

Slice Tests

Focused slices load a subset of application configuration.

```
@WebMvcTest    -> MVC controllers, conversion, advice, selected security
@DataJpaTest    -> JPA repositories and persistence behavior
@JsonTest       -> JSON serialization components
```

Exact included components evolve by Boot version. A slice intentionally excludes many services, so provide explicit test collaborators and do not mistake it for full application wiring.

`@WebMvcTest` can prove request mapping, conversion, validation, controller advice, JSON contracts, and configured security behavior. It cannot prove that the real service implementation is proxied or that the database transaction commits. `@DataJpaTest` can prove mappings and repository queries but does not prove that a controller invokes them correctly.

Avoid importing the main application configuration indiscriminately into a slice, because doing so can silently turn the slice back into a broad context. Import the specific converter, advice, security component, or mapping configuration required by the risk, and keep at least one composition test to prove the complete application.

MockMvc and Real Server Tests

MockMvc exercises Spring MVC without opening a real network socket. It is excellent for mappings, binding, validation, advice, filters configured in the test, and response contracts.

A random-port `@SpringBootTest` exercises embedded server configuration, actual HTTP codecs, network stack, compression, and client behavior. Use it for risks MockMvc cannot represent.

```
MockMvc: fast MVC pipeline fidelity
real port: server and network fidelity
```

MockMvc is not "less real" in every dimension. It invokes the actual MVC mappings, argument resolvers, validation, exception resolvers, filters included in the context, and message converters. It is often the best test for a detailed HTTP contract. It simply does not cross a socket or use the embedded container's real connector.

Use a real server when behavior depends on network boundaries: port and base-path configuration, redirects, cookie handling by a real client, streaming, compression, servlet-container limits, TLS integration, or filter/container dispatch behavior. A few carefully chosen real-server tests usually provide more value than duplicating every controller scenario at both levels.

Transactional Test Traps

Test methods marked transactional commonly roll back after completion. Application code can appear transactional because repository work participates in the test transaction even if the service annotation is absent.

Commit when testing database constraints, after-commit events, async visibility, generated behavior, and lock/isolation cases. Use separate threads or connections for concurrency tests because one test transaction cannot model two concurrent actors.

The first-level persistence context can also hide SQL behavior. Saving an entity and immediately reading it in the same context may return the managed instance without executing the query expected in production. Flush SQL to the database and clear the persistence context before assertions that are supposed to verify mapping or retrieval.

```
repository.save(order);
entityManager.flush();
entityManager.clear();

Order reloaded = repository.findById(order.getId()).orElseThrow();
assertThat(reloaded.getLines()).hasSize(2);
```

Rollback is convenient cleanup, but committed tests need another isolation mechanism: truncate controlled tables, use a fresh schema or container, or create uniquely scoped test data. Cleanup must not race with parallel tests.

Testcontainers and Real Dependencies

Embedded databases differ from production in SQL, locking, indexes, JSON types, query plans, and isolation. Testcontainers can start the actual database, broker, or compatible service in a disposable environment.

```
test process -> disposable PostgreSQL container
               -> run migrations
               -> execute repository/service tests
```

Keep containers reusable at the test-suite level where safe, wait for readiness, and capture logs on failure. Do not replace every unit test with a container test.

Run the same migration tool and scripts used in deployment. Creating tables through ORM auto-generation can let tests pass while the production migration is incomplete. Seed only the minimum scenario data so failures remain understandable; enormous shared fixtures create hidden coupling and slow every test.

Container reuse improves speed but can leak state. Decide whether isolation is per test, class, suite, or worker process, and make that policy explicit. Parallel tests require independent schemas, topics, consumer

groups, or key prefixes where the dependency is shared.

Testcontainers verifies protocol-compatible behavior on the developer or CI host. It does not reproduce production storage latency, cloud identity, network policy, failover, or managed-service configuration. Those risks belong in staging, deployment verification, or controlled resilience exercises.

Dynamic Properties

DynamicPropertySource or supported service-connection features can feed container ports and credentials into the test Environment. Ensure property registration occurs before context creation and avoid static mutable state leaking between suites.

Test configuration should resemble production property names and validation. Disabling security, migrations, or observability globally in tests can hide startup and wiring defects.

Testing Time, Async, and Retries

Inject Clock and schedulers where possible. Avoid `Thread.sleep` for ordering. Complete controlled futures, use latches, and make bounded await assertions.

Retry tests should assert attempts, backoff policy through a controllable scheduler where possible, final failure, idempotency, and transaction boundaries. Fast tests should not wait through production delays.

Flaky tests usually contain an uncontrolled dimension: wall clock, random data, scheduler ordering, shared state, eventual-consistency timing, or an external dependency. Retrying the test suite hides evidence rather than fixing the cause. Capture the seed for generated tests, inject clocks and random sources, and await a meaningful condition with a bounded deadline.

For concurrent code, assert invariants across many interleavings rather than asserting one convenient order. Latches can establish the race window deliberately. Integration tests can prove a specific database conflict, while stress tests and tools such as Java Flight Recorder provide additional evidence under sustained scheduling pressure.

Actuator Architecture

Spring Boot Actuator contributes management endpoints when enabled and permitted. HTTP endpoints normally use an `/actuator` base path, with health commonly available according to exposure and security configuration.

```
application port or management port
/actuator/health
/actuator/metrics
/actuator/prometheus when configured
other explicitly exposed endpoints
```

Endpoint availability has two dimensions: enabled in the ApplicationContext and exposed over a technology such as HTTP or JMX. Do not assume adding the dependency publishes every endpoint.

Management endpoints are application capabilities with their own access policy and operational cost. `health` may be safe for a platform probe with limited details; `heapdump`, `env`, `mappings`, `loggers`, and shutdown operations are materially more sensitive. Enable and expose only what an operational use case requires.

Separate management port and address can simplify network policy, but it creates another listener to configure, secure, probe, and monitor. When the application and management endpoints use different ports, test both external paths through the actual ingress or service configuration.

Health, Liveness, and Readiness

Liveness answers whether the process should be restarted. Readiness answers whether it should receive new traffic.

```
liveness failure -> restart may help
readiness failure -> remove from traffic while process can recover
```

Do not make liveness depend on a remote database or API; a dependency outage can restart every instance and amplify failure. Readiness may include critical dependencies, but consider whether removing all instances is worse than degraded service.

Health details can expose database names, disk paths, or topology. Show details only to authorized management clients.

A health indicator is not a substitute for request metrics. Polling a dependency every second can add load and still miss partial failure between polls. Prefer passive signals for ongoing service health, and keep active checks bounded with short timeouts and no expensive queries.

Readiness policy is a capacity decision. If every instance marks itself unready during a shared database outage, the load balancer has no destination and recovery probes may compete with traffic. Sometimes removing traffic is correct; sometimes serving cached or degraded responses is better. Define readiness from the service's ability to provide its promised minimum behavior, not from a checklist saying every dependency is green.

Metrics with Micrometer

Boot integrates Micrometer as a metrics facade. Useful service metrics include request latency and outcome, JVM/GC, executor queues, connection pools, cache behavior, messaging, and domain outcomes.

Metric tags must have bounded cardinality.

```
good tag: route=/orders/{id}, outcome=SUCCESS
bad tag:  orderId=93b..., userEmail=...
```

High-cardinality labels increase monitoring cost and can exhaust backend memory. Never include secrets or personal data.

Counters describe accumulated events, gauges sample a current value, timers capture duration and count, and distribution summaries capture non-time amounts. Choose the instrument that matches the question. A gauge of completed requests can go down or disappear with the observed object, whereas a counter expresses monotonic event occurrence.

Latency percentiles are more useful than averages for user experience, but local percentiles cannot be aggregated correctly across instances. Histogram buckets allow backend aggregation when bucket boundaries are designed for the service-level objective. Record both latency and outcome; a very fast error is not a successful optimization.

Domain metrics should report decisions that logs cannot summarize cheaply, such as accepted orders, rejected payments by bounded reason, idempotency replays, queue age, or fallback use. Avoid mirroring every log statement as a counter.

Observations and Tracing

Micrometer Observation can connect metrics and tracing instrumentation. Traces show request paths across services; baggage propagates selected context.

Sampling means not every request is retained. Metrics remain necessary for complete rates. Trace IDs belong in logs for correlation, but trace IDs should not become metric labels.

Propagate context through supported HTTP, messaging, and executor instrumentation. Arbitrary asynchronous work can lose trace and security context.

A trace is a tree or graph of timed spans representing one distributed operation. Span attributes explain the operation; events mark notable points; status describes its outcome. Create spans around meaningful boundaries such as an outbound call or message handling, not every private Java method.

Baggage travels across process boundaries and is therefore untrusted and potentially expensive. Allowlist small values, validate length and format, and never place secrets or unrestricted personal data in it. Baggage is not automatically a metric label, and converting values such as customer ID into labels would create dangerous cardinality.

Sampling reduces trace cost but can omit the one request being investigated. Tail-based sampling can retain interesting traces based on completed outcome, at the cost of collector complexity and buffering. Metrics provide complete aggregate detection; exemplars or trace IDs help move from an unusual metric bucket to representative request evidence.

Logging

Use structured logs with timestamp, level, service, instance, trace/correlation ID, event name, and bounded domain identifiers. Avoid logging full request bodies or credentials.

Log configuration changes and level adjustments are operationally sensitive. Runtime logging endpoints should be strongly secured and audited because verbose logging can expose data or overload disk.

Logs should describe events, not narrate every method entry. One request can cross many services, so duplicate stack traces and verbose success logs quickly become cost and signal problems. Log expected business rejection at an appropriate level with stable event code; log an unexpected defect once at the responsible boundary with the causal chain.

Redaction must happen before data reaches the logging sink. A downstream index access rule cannot undo a secret already copied into buffers, agents, archives, or incident exports. Prefer allowlisted fields to brittle regex removal of unknown payloads.

Service-Level Objectives and Alerting

An SLI measures service behavior, such as successful request ratio or latency below a threshold. An SLO states the target over a window. The error budget is the allowed amount of failure. This turns alerting from "CPU exceeded 80 percent" into a user-impact decision.

```
SLI: good requests / eligible requests
SLO: 99.9 percent good over 30 days
budget: 0.1 percent may fail or exceed the latency objective
burn rate: how quickly current failures consume that budget
```

CPU, heap, connection pools, and queue depth remain valuable diagnostic signals, but they are not direct service promises. Alert immediately on rapid error-budget burn and use resource saturation to explain or predict impact. Every alert should have ownership, a runbook, and a clear reason a human must act now.

Securing Actuator

Expose only required endpoints. Place management traffic on a protected network or separate port when appropriate, require authentication and authorization, and apply TLS and firewall policy.

Endpoints for environment, configuration properties, heap dumps, thread dumps, loggers, mappings, and shutdown can disclose or change sensitive state. Sanitization helps but does not replace access control.

Do not expose a heap dump over a broadly reachable endpoint. It can contain credentials and user data.

Graceful Shutdown

Production shutdown should mark the instance unready, stop accepting new work, drain in-flight requests and consumers, close resources, and finish before platform termination.

```
deployment termination
-> readiness false
-> stop new traffic/message intake
-> drain bounded in-flight work
-> close executors, pools, clients
-> process exits
```

Coordinate Boot/server graceful-shutdown settings with Kubernetes termination grace, load-balancer propagation, message visibility timeout, and application lifecycle phases.

Draining is bounded. A process cannot wait forever for a stuck request, and the platform will eventually send a hard termination. Set the application shutdown timeout below the platform grace period, reserve time for cleanup, and ensure clients retry safely when work is interrupted.

Message consumers need protocol-aware shutdown. Stop fetching new records, finish or abandon in-flight work before visibility or session timeouts, commit acknowledgements only after durable processing, and keep idempotency so forced termination does not create duplicate effects.

Startup and Availability

Application availability states distinguish liveness and readiness during startup and shutdown. Startup runners should not accept traffic before required initialization finishes.

Avoid long data migrations inside every application instance. Use controlled migration ownership and compatibility so old and new versions can coexist during rolling deployment.

Production Runbooks

A runbook should define:

- How to identify version, configuration, and active profiles.
- Which dashboards and service-level indicators to inspect.
- How to capture JFR, thread dump, heap evidence, and condition report safely.
- Which Actuator endpoints are allowed and who can access them.
- How to shed load, disable a feature, or roll back.
- What evidence must be preserved before restart.

Automated alerts need actionable thresholds and ownership. Alerting on every transient readiness failure creates noise; alert on sustained user impact and exhausted error budgets.

Runbooks should distinguish evidence collection from mitigation. Restarting may restore service but destroys thread state, in-memory queues, and some profiling evidence. Capture cheap, time-sensitive evidence first when safe, then reduce impact through rollback, traffic shift, feature disablement, load shedding, or restart.

Practice the procedure. An untested heap-dump command may fill the disk; an emergency logging change may expose credentials; a feature flag may not work while the configuration service is unavailable. Game days convert documentation into verified operational capability.

A Strong Interview Answer

I choose the smallest test environment that contains the behavior under risk. Domain rules get direct unit tests, MVC and JSON contracts get slice tests, proxy and wiring behavior gets a Spring context, and database semantics get the production engine through Testcontainers. I explicitly test commit, concurrency, time, retries, and negative security paths rather than relying on test rollback or mocks. In production I expose only required Actuator capabilities, separate liveness from readiness, define bounded-cardinality metrics and SLOs, correlate traces and structured logs, and propagate context only through supported boundaries. Graceful shutdown, alerts, and runbooks are tested parts of the design, not deployment afterthoughts.

Interview-Ready Summary

Test ordinary logic without Spring, use slices for focused framework behavior, and use full-context or real-server tests only for risks requiring them. Transactional tests can hide missing application boundaries, and production-compatible containers reveal database and broker differences.

Actuator management requires explicit enablement, exposure, and security. Liveness should not depend on remote services; readiness controls traffic. Metrics need bounded tags, traces need context propagation, and management endpoints can expose sensitive state. Graceful shutdown coordinates application lifecycle with the deployment platform.

Revision Notes

- Use the smallest test environment that proves the risk.
- Constructor injection enables fast tests without Spring.
- Full context tests verify wiring but cost more.
- Context customization can defeat test-context caching.
- Slice tests intentionally exclude large parts of the application.
- MockMvc tests MVC without a real network socket.
- Random-port tests cover embedded server and HTTP behavior.
- Test transactions can hide missing application transaction boundaries.
- Commit tests when verifying after-commit and database behavior.
- Use production-compatible containers for SQL, locking, and broker semantics.
- Inject time and control asynchronous completion in tests.
- Actuator enablement and exposure are separate decisions.
- Liveness asks whether restart helps.
- Readiness asks whether the instance should receive traffic.
- Do not tie liveness to remote dependency availability.
- Micrometer tags must have bounded cardinality.
- Trace identifiers belong in logs, not metric tags.
- Protect management endpoints with network and identity controls.
- Heap, environment, logger, and shutdown endpoints are sensitive.
- Graceful shutdown drains work within platform deadlines.
- Runbooks preserve evidence and define safe mitigation.